FINE-TUNING BERT

09 November 2026

CSCI 390: Natural Language Processing and Applications | Dr. Qi Li

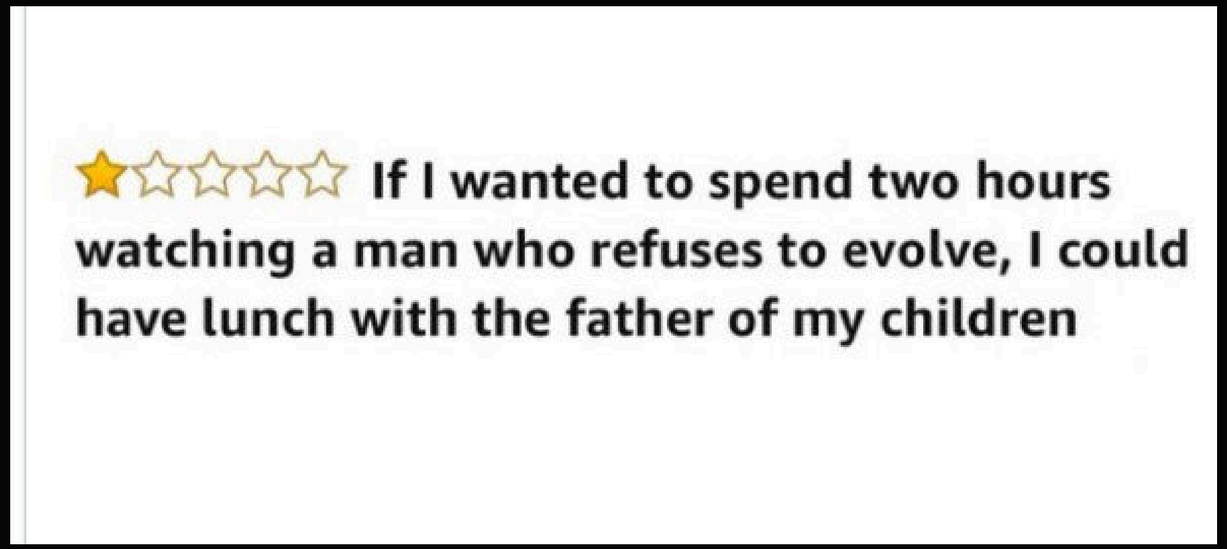
Presented By: Somtochukwu Chukwudi-Edoka and Nicoll Suarez

Sentiment Analysis on Movie Reviews

PROBLEM AND OBJECTIVE

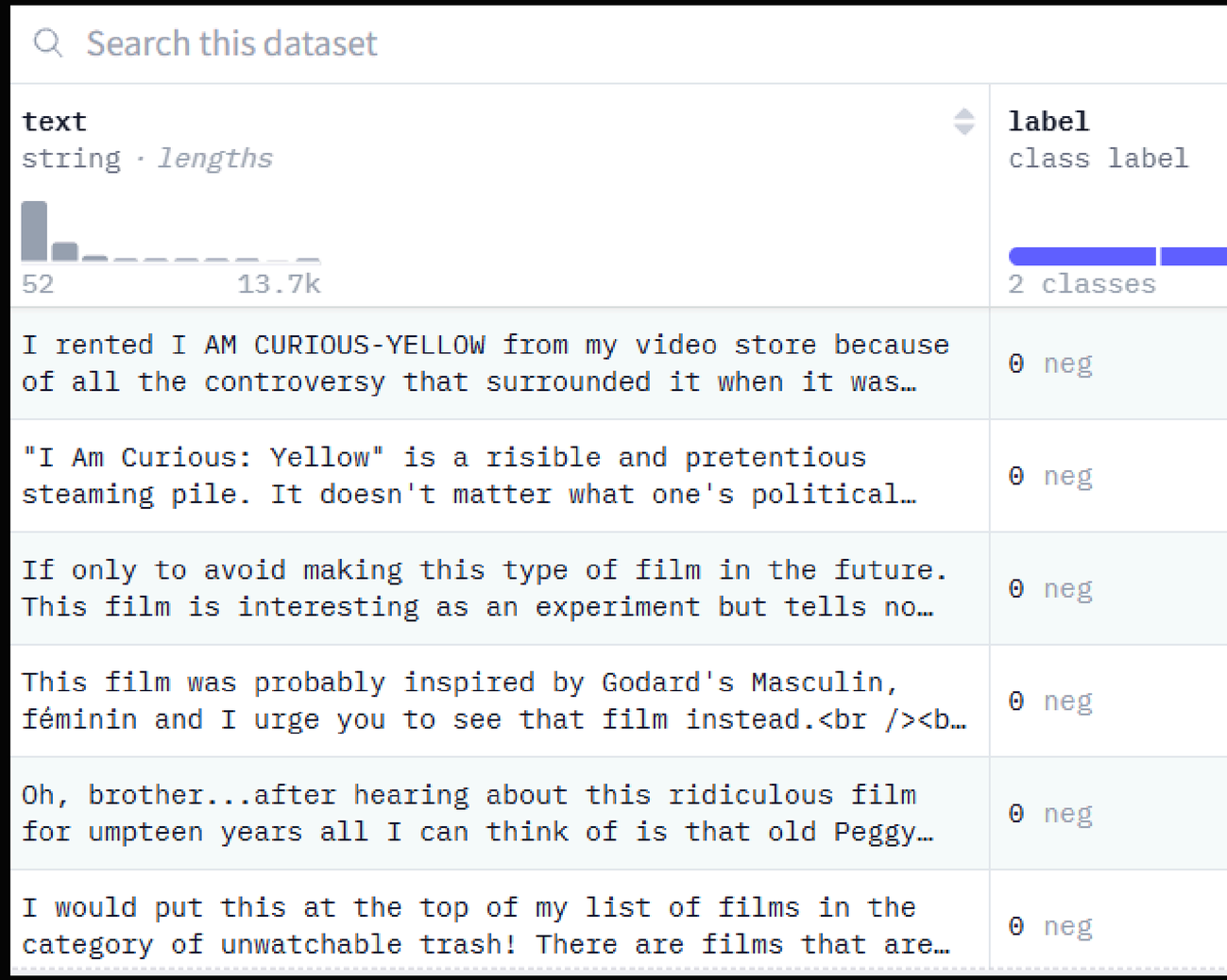
- **Goal:** classify movie reviews as positive or negative
- Reviews are long, messy, unstructured text
- **Approach:** fine-tune a pretrained transformer instead of training from scratch
- **Full pipeline:** preprocess → tokenize → fine-tune → evaluate → live demo





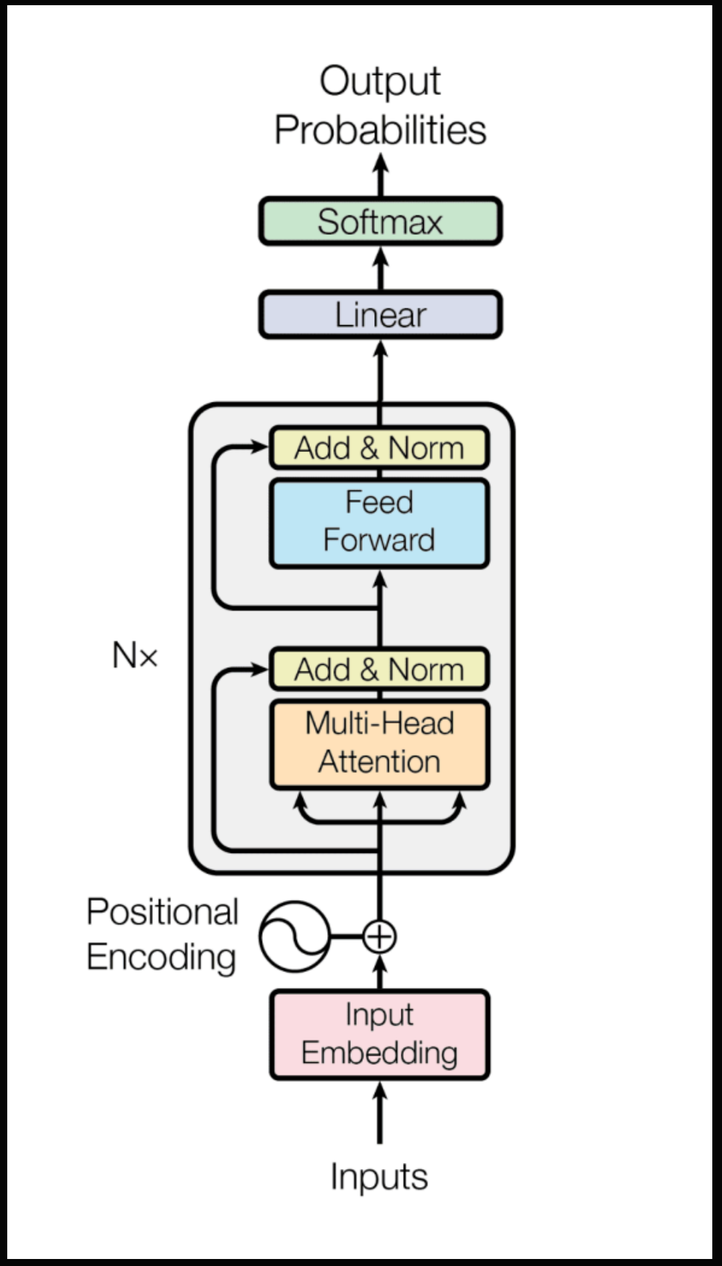
DATASET

- **IMDB Dataset:** 50,000 highly polar movie reviews
- 25,000 for training, 25,000 for testing
- **Balanced labels:** 0 = negative, 1 = positive
- Loaded with the **Hugging Face** datasets library
- Evaluated on a 5,000-review subset of the test split



WHAT IS BERT?

- **B**idirectional **E**ncoder **R**epresentations from **T**ransformers (Google, 2018)
- Reads text in both directions, so each word is understood from its full context
- **bert-base-uncased:** 12 layers, 768 hidden size, 12 attention heads, about 110M parameters
- **“Uncased” =** lowercases all text



PRETRAINING VS. FINE-TUNING

Pretraining

Learns general language from huge text (Wikipedia + books)

Fine-tuning

Adapts the model to one task using labeled data

This is transfer learning: less data, less time, better results

OUR ARCHITECTURE



- The linear classification head is new (randomly initialized)
- All BERT weights are fine-tuned along with the head

CODE WALKTHROUGH: DATA

- Tokenizer converts text into numbers BERT understands
- **Truncation:** reviews cut to 256 tokens
- **Dynamic padding:** each batch is padded only to its longest review (saves memory)

```
ds = load_dataset("stanfordnlp/imdb")  # 25k train / 25k test, labels 0=neg 1=pos


def tokenize(batch):
    # truncation keeps long reviews within BERT's limit; padding is done per-batch
    return tok(batch["text"], truncation=True, max_length=256)


ds = ds.map(tokenize, batched=True)
train_ds = ds["train"].shuffle(seed=42)
# To save time, use a 5k-review validation subset of the test split
val_ds = ds["test"].shuffle(seed=42).select(range(5000))
collator = DataCollatorWithPadding(tok)
```

CODE WALKTHROUGH: MODEL AND TRAINING

```
args = TrainingArguments(
    output_dir="bert-imdb",
    num_train_epochs=2,
    per_device_train_batch_size=16,  # lower to 8 if you hit CUDA out of memory
    per_device_eval_batch_size=32,
    learning_rate=2e-5,
    weight_decay=0.01,
    fp16=True,
    eval_strategy="epoch",
    save_strategy="epoch",
    logging_steps=100,
    report_to="none",
)

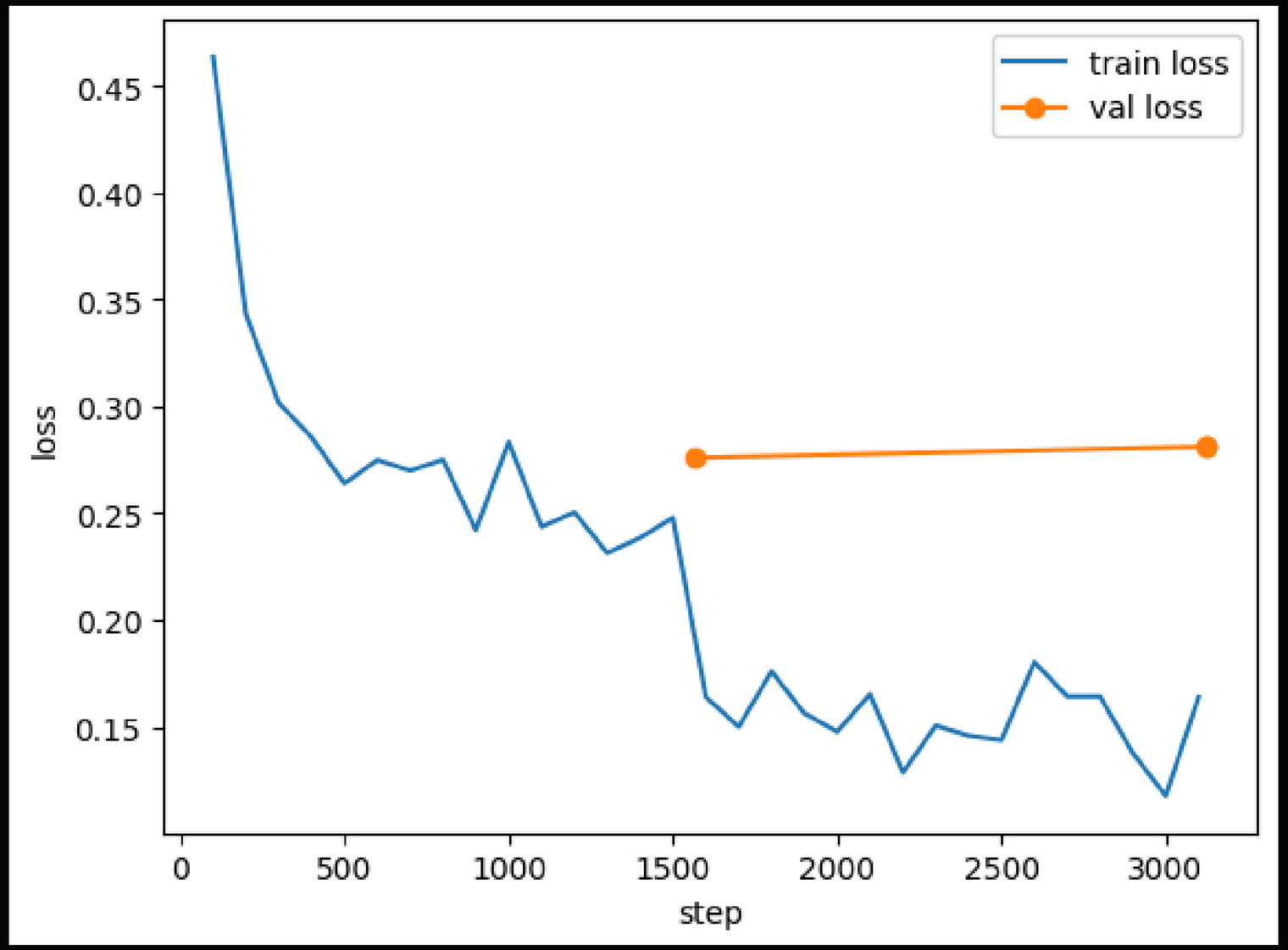
trainer = Trainer(
    model=model,
    args=args,
    train_dataset=train_ds,
    eval_dataset=val_ds,
    data_collator=collator,
    compute_metrics=compute_metrics,
```

- AutoModelForSequenceClassification with 2 labels
- 2 epochs, batch size 16, learning rate 2e-5, fp16 (mixed precision)
- Hugging Face Trainer runs the training loop
- **Metrics:** accuracy and F1 (scikit-learn)
- Trained on a free **Colab T4 GPU**

RESULTS

Epoch	Training Loss	Validation Loss	Accuracy	F1
1	0.247980	0.275975	0.906400	0.911229
2	0.163948	0.281212	0.921000	0.922259

- **Final model:** 92.1% accuracy, F1 0.922
- F1 balances precision and recall, so the model isn't favoring one class



ANALYSIS

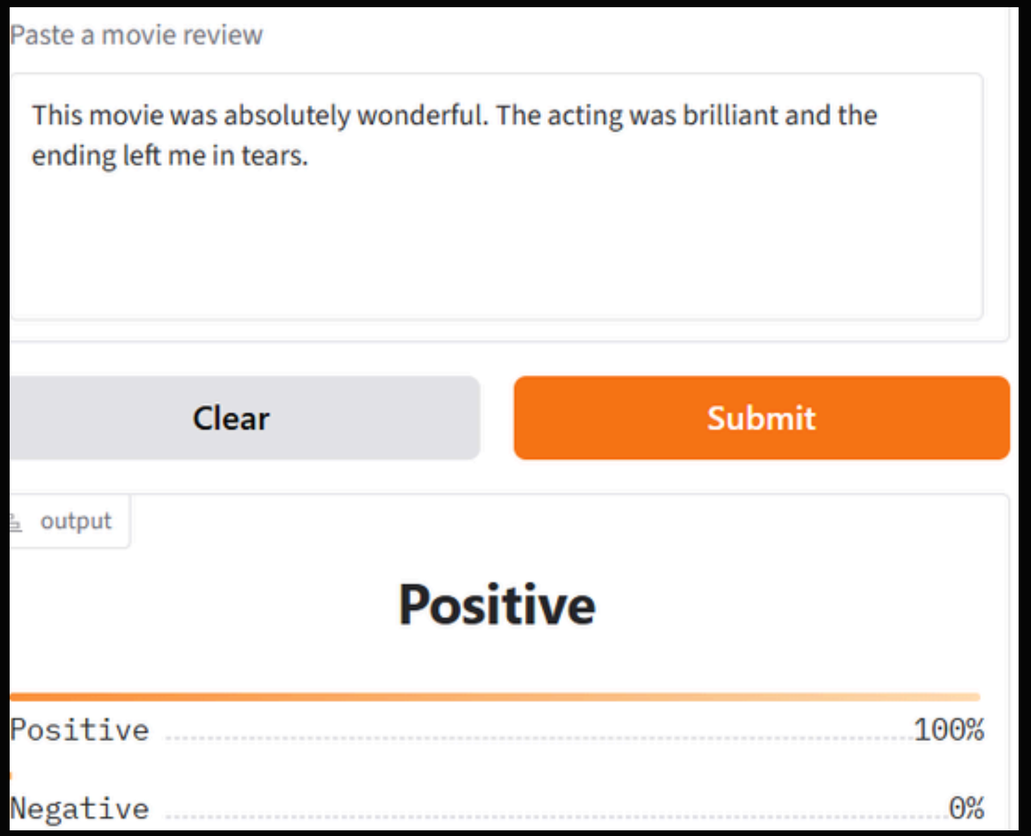
- Training loss fell from 0.25 to 0.16
- Validation loss stayed flat (0.276 → 0.281)
- This is mild overfitting: the model begins memorizing training data
- Accuracy still improved, so 2 epochs was a good stopping point
- More epochs would likely not help

CHALLENGES AND DEBUGGING

- **Dataset loading error:** HfUriError. Fixed by using the full name stanfordnlp/imdb
- **Lost model:** Colab session reset and wiped the saved model. Fixed by saving to Google Drive and retraining
- **Variable review lengths:** solved with truncation + dynamic padding
- **Demo error:** “Could not parse server response” after the session died. Fixed by reloading and relaunching

LIVE DEMO

Paste a movie review → get a sentiment prediction with confidence





LIMITATIONS

- Sarcasm is hard for the model
- Reviews over 256 tokens get cut off
- Only tested on English movie reviews

FUTURE WORK

- Try other models like RoBERTa or DistilBERT
- Tune hyperparameters
- Test on other datasets

CONCLUSION

- Fine-tuning BERT reached about 92% accuracy
- Fast training on a T4 GPU

THANK YOU.

09 November 2026

CSCI 390: Natural Language Processing and Applications | Dr. Qi Li

Presented By: Somtochukwu Chukwudi-Edoka and Nicoll Suarez